Comparison of Matlab/Python implementations of Single Hidden Layer Perceptron with Backpropagation on Dry Bean dataset

Data, Summaries and Preprocessing

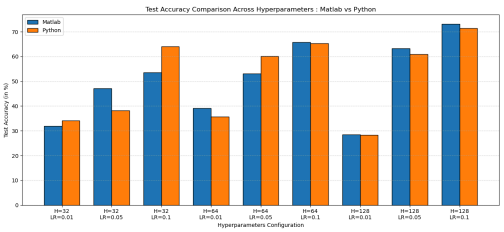
The dataset chosen for this study was the Dry bean Dataset (7 Classes, 13,611 Samples, 16 Features), from the UCI ML Repository [1]. The aim was Multiclass Classification of Bean varieties using a Single Hidden Layer MLP [2]. The dataset was obtained with no missing values or anomalies. Stratified sampling was used with a 80% Training (10,889) , 20% Testing Split (2,722). Feature data was non categorical and was scaled using z-score normalisation. Target Categorical Data Labels were one-hot encoded.

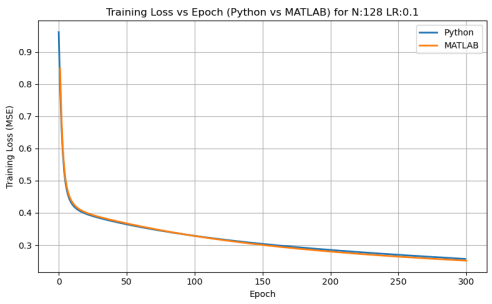
Implementation

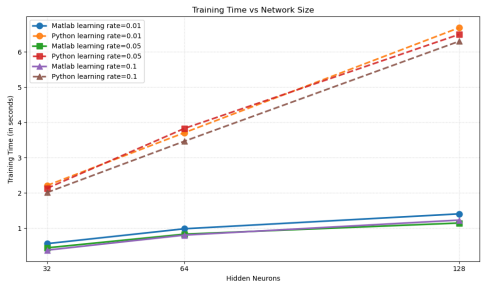
A Single hidden layer MLP with 16 input neurons and 7 output neurons was trained with varying hidden layer sizes. Weights were initialised and scaled using Normalised Initialization (Xavier/Glorot - Random and Scaled). For Training - Sigmoid activation functions were used for the hidden layer and output layer and the loss function used was Mean Squared Error (MSE). It should be noted that it would have been more optimal to use SoftMax with Cross Entropy however the goal of this project was not optimization. Standard Backpropagation was used, manually implemented in both Python and Matlab (no momentum/regularization). Several variations of hyperparameters were tested.

Hyperparameters Tested:

- ***Hidden Neurons*** [32, 64, 128] / ***Learning Rates*** [0.01, 0.05, 0.1] / ***Epochs*** 300







Results and Conclusion

Hyperparameter variations produce similar trends in accuracy for both Python and Matlab, improving from ~30% (H=32, LR=0.01) to ~70% (H=128, LR=0.1), accuracy improves with larger networks and higher learning rates. Implementation accuracies were closer for larger networks, smaller networks showed higher variance - this is consistent with smaller networks having higher sensitivity to random weight initialization [3].

Training Loss vs Epoch for the optimal model (H=128, LR=0.1, A=~70%) shows overlapping loss curves validating implementation equivalence of forward and backpropagation within both Matlab and Python - as a result of manual coding using identical methods.

Training time increases linearly with network size. A clear conclusion is that Python implementation is significantly slower than Matlab, with peaks of ~6.5s as opposed to ~1.5s. However it can be noted that learning rate has minimal impact on training time as shown by the grouping of learning rates being tightly clustered.

[1]https://archive.ics.uci.edu/dataset/602/dry+bean+dataset
[2]https://www.semanticscholar.org/paper/Multiclass-classification-of-dry-beans-using-vision-Koklu-%C3%96zkan/e84c31138f2f261d15517d6b6bb8922c3fe597a1
[3]https://proceedings.mlr.press/v80/safran18a.html